Divisible by K Pattern Notes

Prefix Sum + Modulo and DP + Modulo

Master Pattern Identification Sheet

The first thing to identify is:

```
Is it a Subarray problem or a Subset problem?
```

Rule 1

Keywords

```
Subarray
Continuous
Contiguous
Range
```

Think:

```
Prefix Sum
```

because:

```
sum(i...j)
=
prefix[j] - prefix[i-1]
```

Rule 2

Keywords

```
Subset
Subsequence
Pick / Don't Pick
```

Think:

```
Take / Not Take DP
```

because elements are not contiguous.

Rule 3

Keywords

```
Divisible by K
Multiple of K
Modulo
Remainder
```

Think:

```
sum % k
```

You usually don't need the actual sum.

Final Identification Table

Question Type	First Thought
Subarray Sum	Prefix Sum
Continuous Subarray	Prefix Sum
Count Subarrays	Prefix Sum + HashMap
Subarray Divisible by K	Prefix Sum + Modulo
Subset Sum	DP
Count Subsets	DP

Question Type	First Thought
Subset Divisible by K	DP + Modulo
Subset/Partition Problems	DP

One-Line Template

```
Contiguous → Prefix Sum
Non-Contiguous → DP
Divisible by K → Modulo
```

Pattern Tree

```
Divisible by K
        ↓
      Modulo
        ↓
  ┌─────────────┐
  │             │
Subarray      Subset
  │             │
Prefix Sum     DP
```

Problem 1

Continuous Subarray Sum

Leetcode 523

Problem Statement

Given an integer array `nums` and an integer `k`, return `True` if there exists a **continuous subarray of size at least 2** whose sum is divisible by `k`.

Brute Force

Generate all subarrays.

```
for i in range(n):
    sm = 0
    for j in range(i, n):
        sm += nums[j]

        if j-i+1 >= 2 and sm % k == 0:
            return True
```

Complexity:

```
O(n²)
```

Main Observation

Suppose:

```
prefix[j] % k
=
prefix[i] % k
```

Then:

```
(prefix[j] - prefix[i]) % k = 0
```

which means:

```
sum(i+1...j)
```

is divisible by k.

What do we need?

For every remainder:

```
Have we seen this remainder before?
```

YES → A valid subarray exists.

Why store first index?

Because subarray length must be:

```
>= 2
```

Data Structure

```
remainder -> first occurrence index
```

Algorithm

1. Compute prefix sum.
2. Compute remainder.
3. If same remainder was seen before:
    - check length.
4. Otherwise store index.

Code

```
def checkSubarraySum(nums, k):

    mp = {0: -1}
    prefix = 0

    for i, num in enumerate(nums):

        prefix += num
        rem = prefix % k
```

```
        if rem in mp:

            if i - mp[rem] >= 2:
                return True

        else:
            mp[rem] = i

    return False
```

Complexity

```
Time  : O(n)
Space : O(k)
```

Pattern Learned

```
Existence Question
+
Subarray
+
Divisible by K
↓
Prefix Sum + First Occurrence Map
```

Problem 2

Subarray Sums Divisible by K

Leetcode 974

Problem Statement

Count the number of subarrays whose sum is divisible by k.

Brute Force

Generate all subarrays.

```
O(n²)
```

Main Observation

Again:

```
prefix[j] % k
=
prefix[i] % k
```

Then:

```
subarray(i+1...j)
```

is divisible by k.

Difference from Problem 523

523:

```
Need existence
```

974:

```
Need count
```

Key Observation

Suppose current remainder:

```
rem = 3
```

and remainder 3 has already appeared:

```
4 times
```

Then current index creates:

```
4 new valid subarrays.
```

Therefore we need:

```
remainder -> frequency
```

Algorithm

1. Compute prefix sum.
2. Compute remainder.
3. Add frequency of this remainder to answer.
4. Increment frequency.

Code

```
def subarraysDivByK(nums, k):

    cnt = {0: 1}

    prefix = 0
    ans = 0

    for num in nums:

        prefix += num
        rem = prefix % k

        ans += cnt.get(rem, 0)
```

```
        cnt[rem] = cnt.get(rem, 0) + 1

    return ans
```

Complexity

```
Time  : O(n)
Space : O(k)
```

Pattern Learned

```
Count Question
+
Subarray
+
Divisible by K
↓
Prefix Sum + Frequency Map
```

Problem 3

Count of Subsets Whose Sum is Divisible by K

Problem Statement

Given an array and integer k, count the number of **non-empty subsets** whose sum is divisible by k.

Brute Force

Subset means:

```
Take
```

```
Not Take
```

Recursion:

```
f(i, sum)
```

Complexity:

```
O(2^n)
```

Main Observation

We don't need:

```
sum
```

We only need:

```
sum % k
```

because the problem only asks:

```
sum divisible by k
```

State Compression

Instead of:

```
f(i, sum)
```

use:

```
f(i, rem)
```

where:

```
rem = sum % k
```

Recurrence

At index `i`:

Don't Take

```
notTake = f(i+1, rem)
```

Take

```
newRem = (rem + arr[i]) % k

take = f(i+1, newRem)
```

Final Recurrence

Since we are counting:

```
return take + notTake
```

Base Case

```
if i == n:
    return 1 if rem == 0 else 0
```

This counts:

```
empty subset
```

too.

Non-Empty Subset Modification

Subtract:

```
answer = f(0,0) - 1
```

because:

```
empty subset sum = 0
```

DP State

```
dp[i][rem]
```

Meaning:

> Number of ways to obtain remainder `rem` from index `i`.

Dimensions

```
Index      : n
Remainder  : k
```

Total:

```
O(n*k)
```

Memoization Code

```
def countSubsetDivisible(arr, k):

    n = len(arr)

    dp = [[-1] * k for _ in range(n)]

    def f(i, rem):

        if i == n:
            return 1 if rem == 0 else 0

        if dp[i][rem] != -1:
            return dp[i][rem]

        take = f(
            i + 1,
            (rem + arr[i]) % k
        )

        notTake = f(
            i + 1,
            rem
        )

        dp[i][rem] = take + notTake

        return dp[i][rem]

    return f(0, 0) - 1
```

Space Optimized DP

Let:

```
dp[r]
```

denote:

```
Number of subsets giving remainder r.
```

Transition

For every number:

1. Start with old states.
2. Add current number to all previous states.

Code

```
def countSubsetDivisible(arr, k):

    dp = [0] * k
    dp[0] = 1

    for num in arr:

        ndp = dp[:]

        for rem in range(k):

            newRem = (rem + num) % k

            ndp[newRem] += dp[rem]

        dp = ndp

    return dp[0] - 1
```

Complexity

```
Time  : O(n*k)
Space : O(k)
```

Pattern Learned

```
Count Question
+
Subset
+
```

```
Divisible by K
↓
DP + Modulo State Compression
```

Complete Pattern Sheet

Subarray + Divisible by K

Think:

```
Prefix Sum + Modulo
```

Existence

```
Store first occurrence.
```

```
remainder -> first index
```

Example:

```
Leetcode 523
```

Count

```
Store frequency.
```

```
remainder -> count
```

Example:

```
Leetcode 974
```

Subset + Divisible by K

Think:

```
Take / Not Take DP
+
Modulo State Compression
```

Existence

Store:

```
Set of possible remainders.
```

Count

Store:

```
Count of ways for each remainder.
```

Final Cheat Sheet

```
Subarray + Sum
        ↓
Prefix Sum

Subarray + Divisible by K
        ↓
Prefix Sum + Modulo

Count Subarrays
        ↓
Prefix Sum + Frequency Map
```

```
Subset + Sum
        ↓
DP

Subset + Divisible by K
        ↓
DP + Modulo

Count Subsets
        ↓
DP Counting
```

Ultimate Interview Rule

```
Contiguous
↓
Prefix Sum

Non-Contiguous
↓
DP

Divisible by K
↓
Modulo

Existence
↓
Boolean / First Occurrence

Count
↓
Frequency / DP Counting
```